

CS 214: Data Structures

Trees

Slide contents follow

Kruse and Leung “Data Structures & Program Design in C”

Prepared by:

Waleed A. Yousef, Ph.D.

Binary Trees (BT) and Binary Search Trees (BST) (Ch. 9)

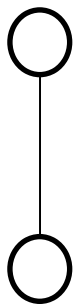
As defined for the analysis of Binary Search:

Definition: A binary tree is either empty, or it consists of a **node** (vertex) called the root together with two binary trees called the left subtree and the right subtree of the root.

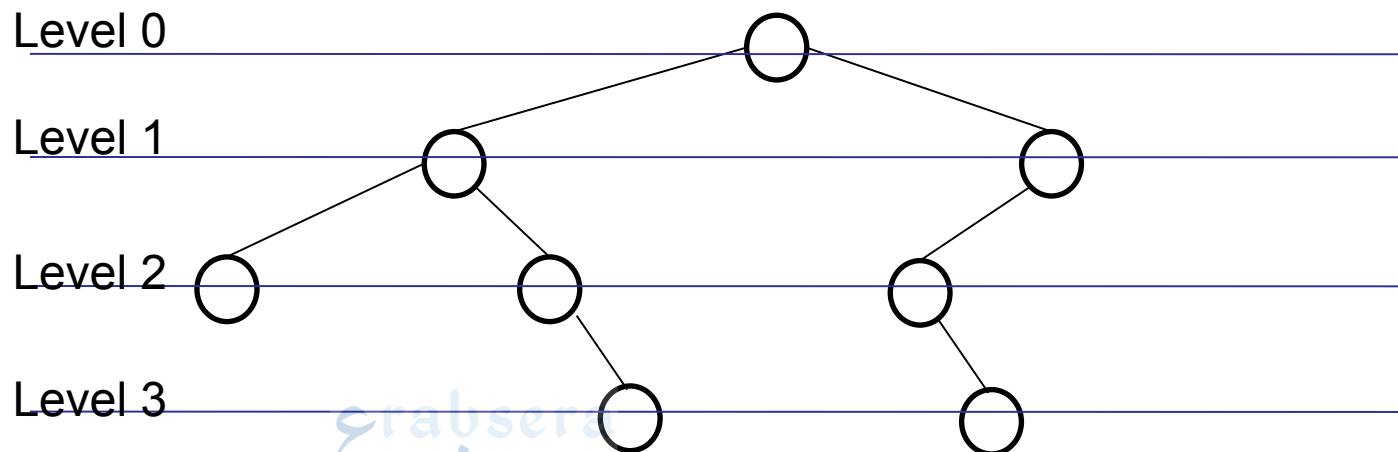
Motivation:

We studied BT (along with the above definition) as a mathematical structure NOT as ADT (because we did not define operations on it!). Why do we need to represent BT as a Data Structure?

There are Multi-way trees (degree higher than two) which will not be taught in our introductory course.



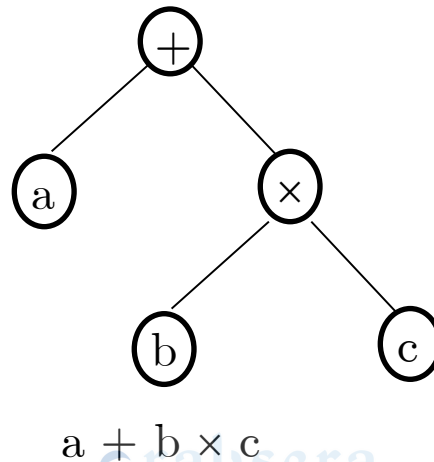
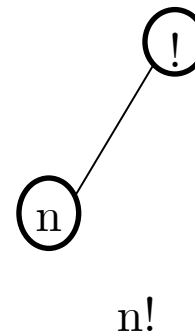
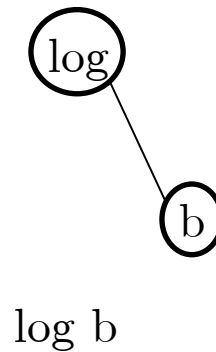
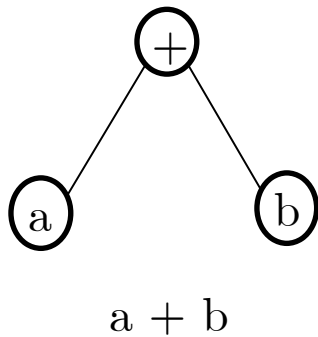
This is not a BT



Motivation for BT

1- If we have to represent the data in a linked structure, i.e., linked list, (because we cannot anticipate the maximum number of elements), while keeping the search fast. Notice that: this idea is immediate after our analysis to the binary search in arrays.

2- Building Expression Trees (has great importance in syntactical analysis and parsing), along with the validity of expressions



Before ADT and coding, How to traverse a BT? More motivations and benefits

There was no problem for traversing a linear structure like a list. For a BT, at each node V having left and right subtrees L and R (respectively) we can do the following visiting:

VLR

Preorder

VRL

LVR

Inorder

LRV

Postorder

RVL

RLV

The standards are Pre, In, and Post order. In the three of them, L precedes R; then V is before them (Pre) or in between (In) or after them (Post).

Example:

Preorder:

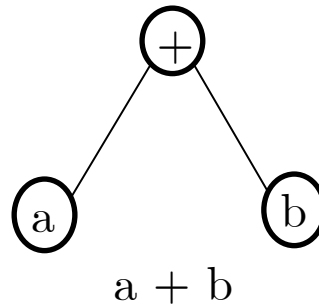
+ab

Inorder:

a+b

Postorder:

ab+



The *Polish Forms* (Ch 12) are related to these orders.

Preorder $\rightarrow$ *Prefix*

Inorder $\rightarrow$ *Infix*

Postorder $\rightarrow$ *Postfix*

Example:

Preorder:

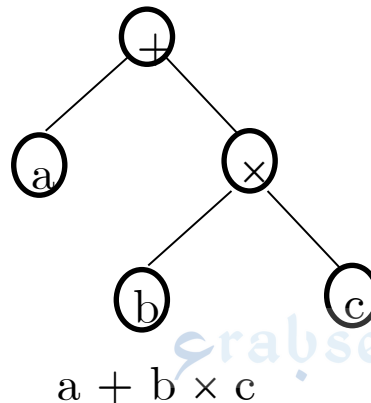
+a×bc

Inorder:

a+b×c

Postorder:

abc×+

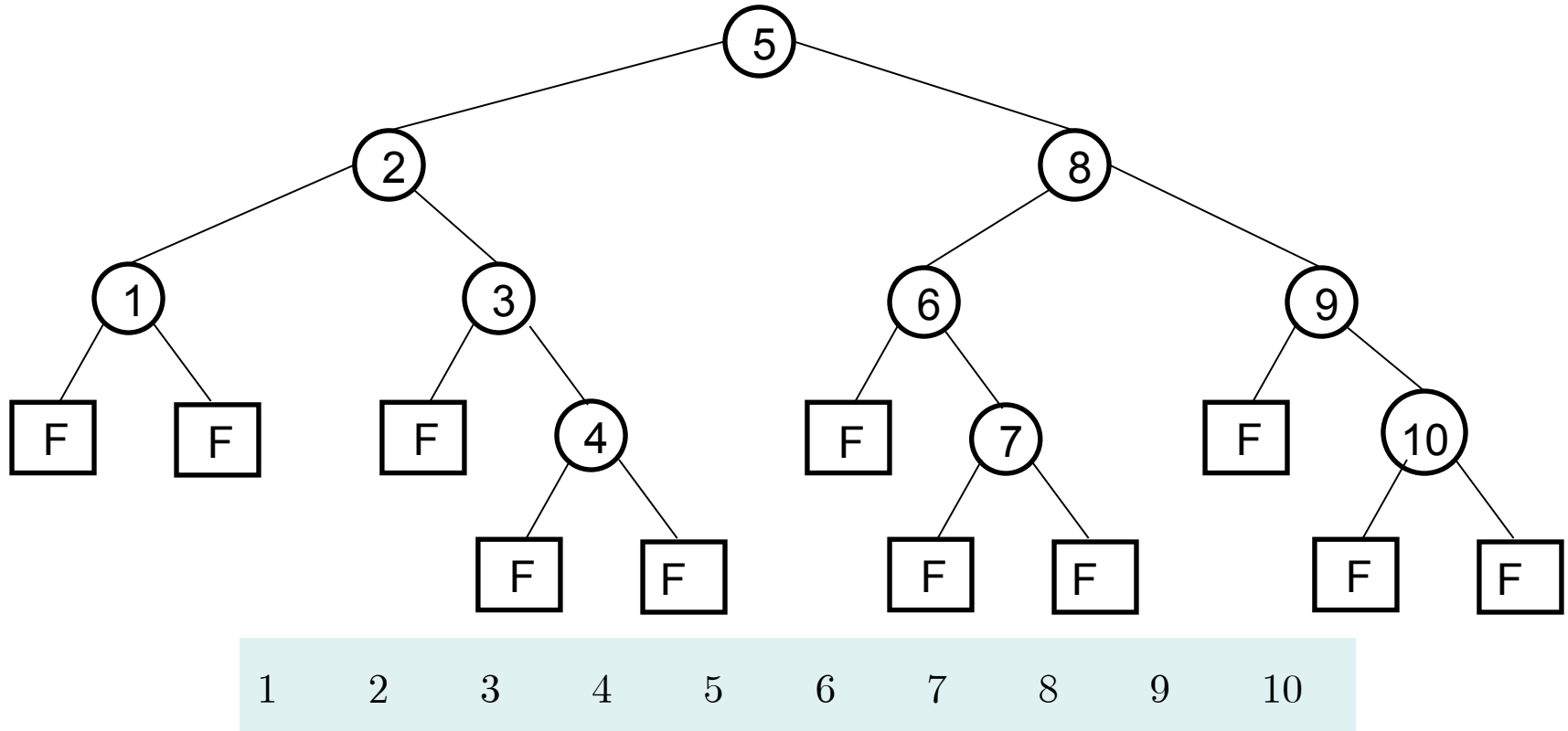


Also notice the strong connection between these traversal modes on a hand and recursion and stacks on the other hand.

Before ADT and coding, How to traverse a BT? More motivations and benefits

Notice that:

For a BT produced by binary search, the Inorder traversal produces sorted elements.



Now, we have enough motivation for having BTs as data structures. Let us define the ADT and start coding as linked implementation.

Definition: A **Binary Tree ADT** is either empty, or it consists of a **node** (vertex) called the root together with two binary trees called the left subtree and the right subtree of the root. This is together with the following operations:

1. Create the tree, leaving it empty.
2. Determine whether the tree is empty or not
3. Determine whether the tree is full or not
4. Find the size of the tree.
5. Traverse the tree, visiting each entry
6. Clear the tree to make it empty

We will define the next operations for a special type of BT, i.e., Binary Search Trees (which will be defined later).

1. Insert a new entry (we have to define where)
2. Delete an entry (we have to define from where)
3. Search for an element.
4. (Any other operation to be defined later).

Linked Implementation

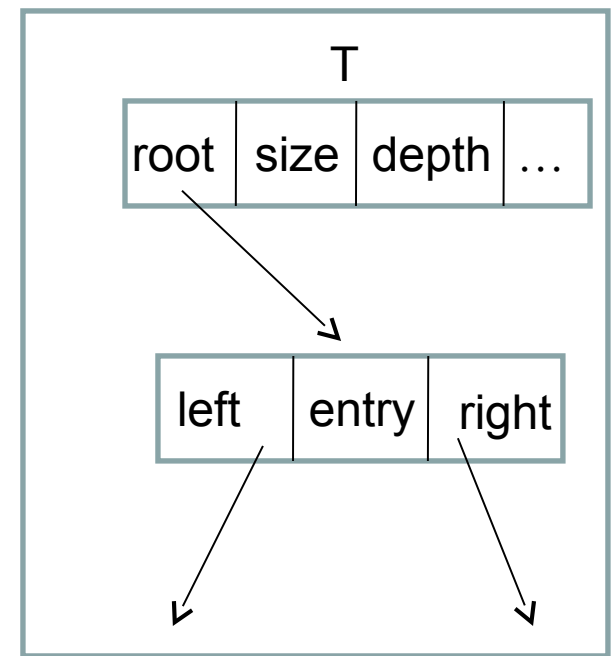
```
/*Tree.h*/
#include "Global.h"

typedef struct treenode{
    TreeEntry entry;
    struct treenode *left, *right;
}TreeNode;

typedef TreeNode * Tree; //Book def.

typedef struct tree{
    TreeNode *    root;
    int           size;
    int           depth;
    //Other fields are possible if needed
}Tree2; //Our def.

void CreateTree    (Tree *);
void ClearTree     (Tree *);
int  TreeSize      (Tree *);
Int  TreeDepth     (Tree *);
void Preorder      (Tree *, void (*) (TreeEntry));
void Inorder       (Tree *, void (*) (TreeEntry));
void Postorder     (Tree *, void (*) (TreeEntry));
```



Notice that: the book defined the tree to be a pointer to a node directly rather than a struct that contains a pointer (along with other fields) to a node. The book itself did not agree with that style before (see page 86).

Study both implementations.

```
void CreateTree(Tree *pt) {
    *pt=NULL;
}
```

```
void CreateTree2(Tree2 *pt) {
    pt->root=NULL;
    pt->depth=0;
    pt->size=0;
}
```

```
int TreeEmpty(Tree *pt) {
    return (!*pt);
}
```

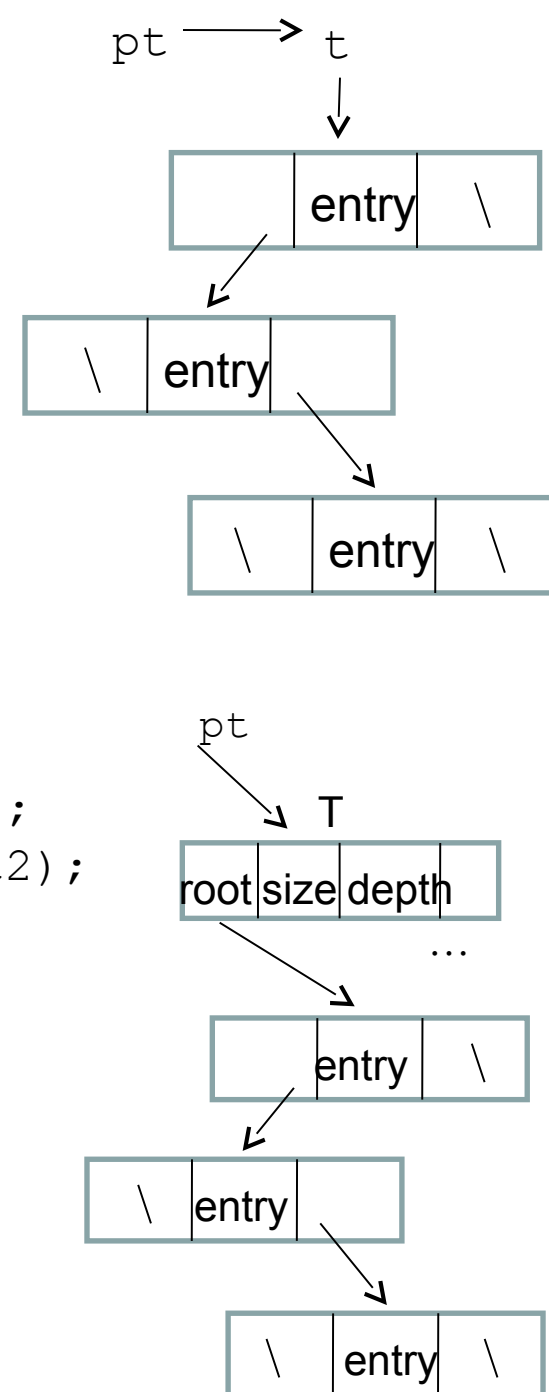
```
int TreeEmpty2(Tree2 *pt) {
    return (!pt->root);
}
```

```
int TreeFull(Tree *pt) {
    return 0;
}
```

```
int TreeFull2(Tree2 *pt) {
    return 0;
}
```

//User level

```
Tree t;
Tree2 t2;
CreateTree(&t);
CreateTree2(&t2);
```



//Pre: Tree has been created and initialized.

//Post: Inorder traversal.

```
void InorderRec(Tree *pt, void(*pvisit)(TreeEntry)){
    if (*pt){
        InorderRec(&(*pt)->left, pvisit);
        (*pvisit)((*pt)->entry);
        InorderRec(&(*pt)->right, pvisit);
    }
}
```

Notice that, we pass a pointer to the tree to be consistent with other functions.

//User level

Tree t;

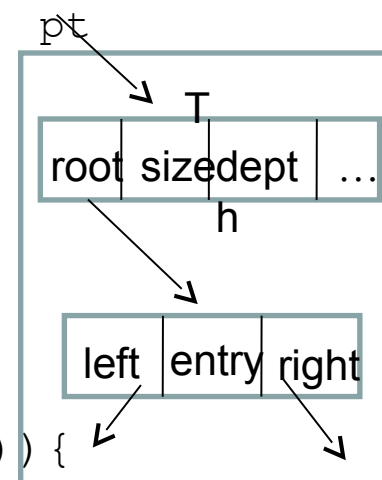
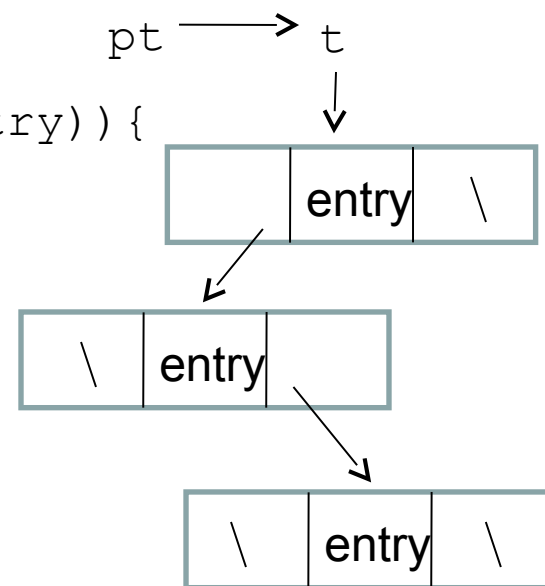
InorderRec(&t, &Display)

```
void InorderRec2(Tree2 *pt, void (*pvisit)(TreeEntry)){
    InorderRec(&pt->root, pvisit);
}
```

//User level

Tree2 t2;

InorderRec2(&t2, &Display)




```

void ClearTreeRec(Tree *pt) {
    if (*pt) {
        ClearTreeRec(&(*pt)->left);
        ClearTreeRec(&(*pt)->right);
        free(*pt);
        *pt=NULL;
    }
}

```

```

//User level
Tree t;
ClearTreeRec(&t);

```

```

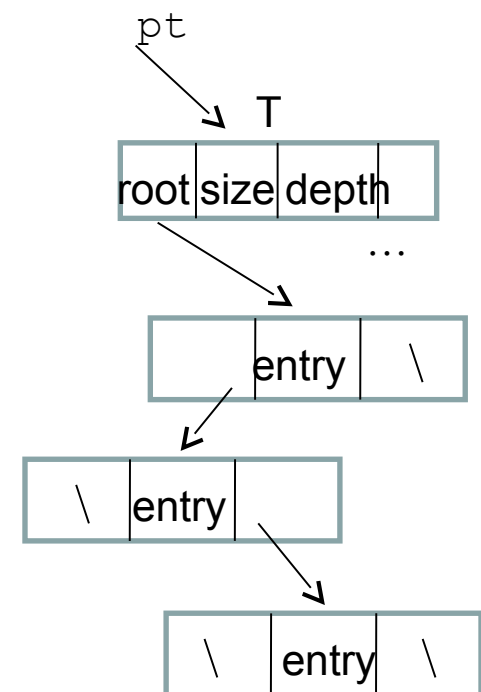
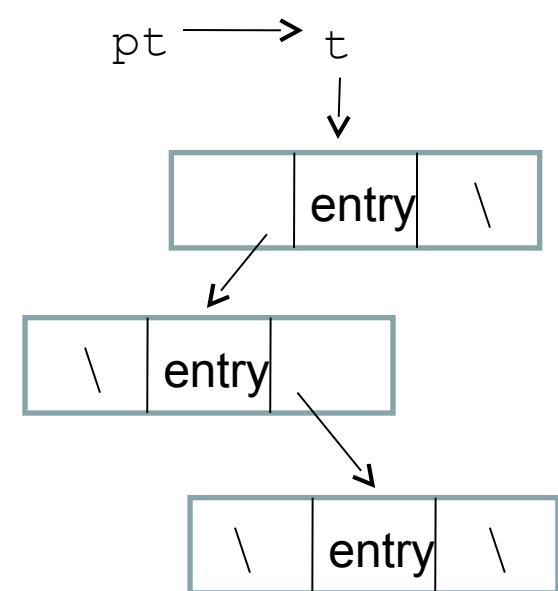
void ClearTreeRec2(Tree2 *pt) {
    ClearTreeRec(&pt->root);
    pt->size=0;
    pt->depth=0;
}

```

```

//User level
Tree2 t2;
ClearTreeRec2(&t2);

```



```
void ClearTreeRec(Tree *pt) {
    if (*pt) {
        ClearTreeRec(&(*pt)->left);
        ClearTreeRec(&(*pt)->right);
        free(*pt);
        *pt=NULL;
    }
}
```

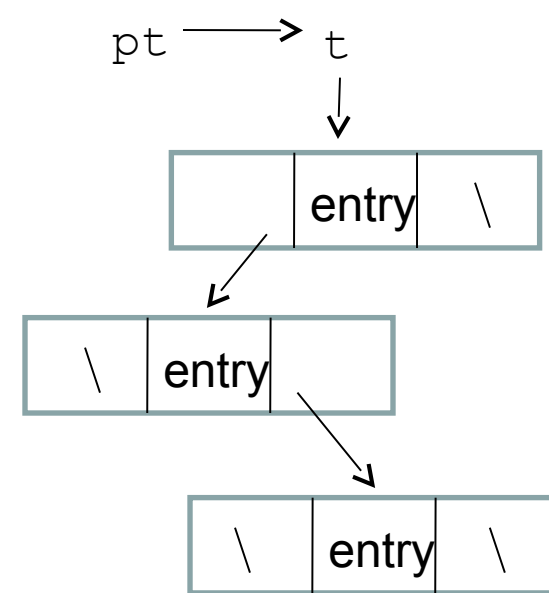
```
void ClearTreeRecAux(Tree *pt) {
    if (*pt) {
        ClearTreeRecAux(&(*pt)->left);
        ClearTreeRecAux(&(*pt)->right);
        free(*pt);
    }
}
```

//of course, this is not provided for the user.

It is very smart to notice that (as was noticed by some students during the lecture)

ClearTreeRecAux is faster than ClearTreeRec because it saves the step `*pt=NULL` in each recursive call. However it does not set the tree to NULL at the end. We can rewrite a faster version of ClearTreeRec as:

```
void ClearTreeRec(Tree *pt) {
    ClearTreeRecAux(pt);
    *pt=NULL;
}
```



Binary Search Trees (BST)

Motivation: To be able to search efficiently in lower complexity than the linked lists.

Definition: A **Binary Search Tree ADT** is either empty, or it consists of a **node** (vertex) called the root together with two binary trees called the left subtree and the right subtree of the root. **Every node contains a key and satisfies the following conditions:**

- The key in the left child of a node (if it exists) is less than the key in its parent
- The key in the right of a node (if it exists) is greater than the key in its parent node.

This is together with the following operations:

1. Insert a new entry (we have to define where)
2. Delete an entry (we have to define from where)
3. Search for an element.
4. (Any other operation to be defined later).

Note:

- The red statement is the only addition to the definition of the Binary Trees.
- The `typedef` part will be exactly the same as the binary trees, along with all the previous functions (since they were common for any binary tree).
- We can simply use: `typedef ListEntry TreeEntry;`
- We will write the functions with a little variation than the book; **you have to study both ways and compare the different programming style.**

//Pre: Tree has been created and initialized.

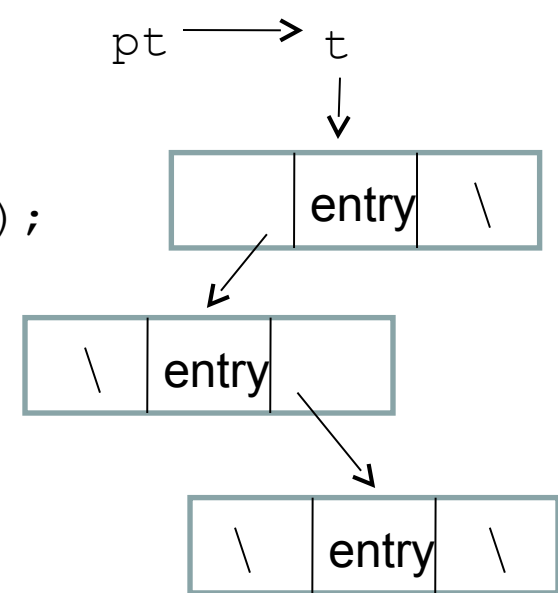
```
void InsertTreeRec(Tree *pt, TreeEntry *pe){
    if (!*pt){
        *pt=(Tree)malloc(sizeof(TreeNode));
        (*pt)->entry=*pe;
        (*pt)->left=NULL;
        (*pt)->right=NULL;
    }else if (LT(pe->key, (*pt)->entry.key))
        InsertTreeRec(&(*pt)->left, pe);
    else
        InsertTreeRec(&(*pt)->right, pe);
}
```

//User level

Tree t;

TreeEntry e;

InsertTreeRec(&t, &e);



Important issues:

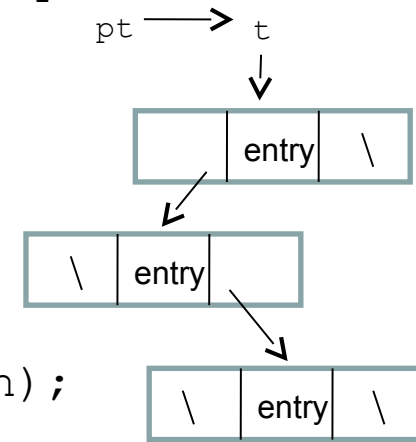
- pe is pointer to element rather than the element itself to save space in recursion.
- malloc code is inside the if statement not at the beginning of the function for same reason.
- Equal keys are inserted to the right. Therefore, Inorder traversal will visit duplicate keys in the same order of insertion.

-Now, how to write this function for Tree2? (We need to account for size and depth!); We need a similar function that returns at which depth the node is inserted.

```

void InsertTreeAux(Tree *pt, TreeEntry *pe, int *pdepth) {
    if (!*pt) {
        *pt=(Tree)malloc(sizeof(TreeNode));
        (*pt)->entry=*pe;
        (*pt)->left=NULL;
        (*pt)->right=NULL;
    } else if (LT(pe->key, (*pt)->entry.key))
        InsertTreeAux(&(*pt)->left, pe, pdepth);
    else
        InsertTreeAux(&(*pt)->right, pe, pdepth);
    (*pdepth)++; //This is the only difference from previous
} //of course, this is not provided for the user.

```

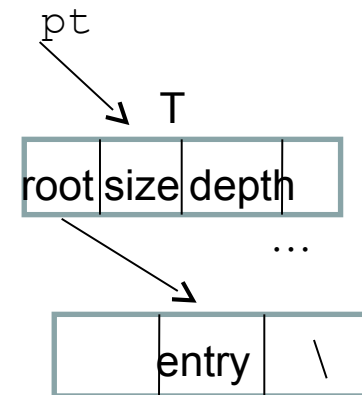


Note: We pass a pointer to int, i.e., pdepth, so that every recursive call increments the same variable.

```

void InsertTreeRec2(Tree2 *pt, TreeEntry *pe) {
    int d=0;
    InsertTreeAux(&pt->root, pe, &d);
    if (pt->depth<d)
        pt->depth=d;
    pt->size++;
}

```



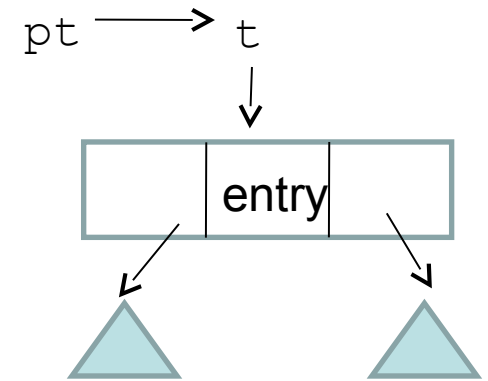
Now, to get the size and depth:

```
int TreeSizeRec(Tree *pt) {  
    if (!*pt)  
        return 0;  
    return (1+TreeSizeRec(&(*pt)->left)+TreeSizeRec(&(*pt)->right));  
}
```

```
int TreeSize2(Tree2 *pt) {  
    return (pt->size);  
}
```

```
int TreeDepthRec(Tree *pt) {  
    if (!*pt)  
        return 0;  
    int a=TreeDepthRec(&(*pt)->left);  
    int b=TreeDepthRec(&(*pt)->right);  
    return (a>b)? 1+a : 1+b;  
}
```

```
int TreeDepth2(Tree2 *pt) {  
    return (pt->depth);  
}
```



Which is better; the previous or any of these two?

```
int TreeDepthRec(Tree *pt) {
    if (!*pt)
        return 0;
    return (1+Max(
        TreeDepthRec(&(*pt)->left), TreeDepthRec(&(*pt)->right));
}

int Max(int x, int y) {
    if (x>y) return x;
    else return y;
}

int TreeDepthRec(Tree *pt) {
    if (!*pt)
        return 0;
    if (TreeDepthRec(&(*pt)->left) > TreeDepthRec(&(*pt)->right))
        return 1+TreeDepthRec(&(*pt)->left)
    else
        return 1+TreeDepthRec(&(*pt)->right);
} //This is the most stupid version
```

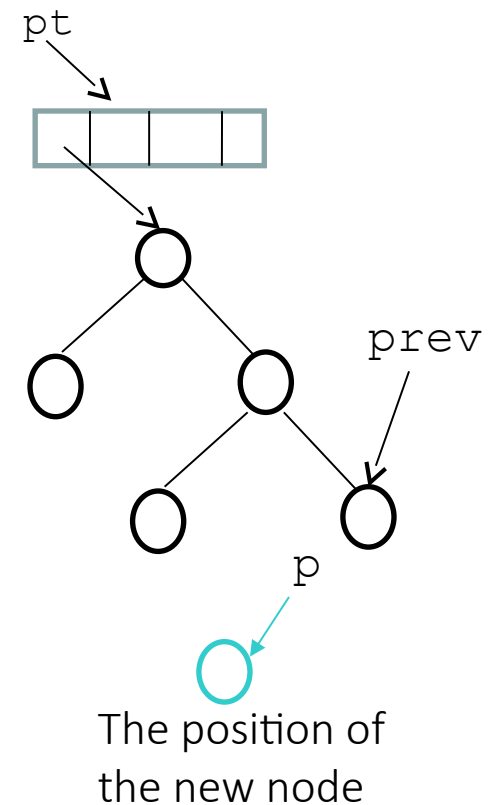
Now, how to insert iteratively? Was there any need for recursion?



```

void InsertTree2 (Tree2 *pt, TreeEntry *pe) {
    TreeNode *p, *prev, *curr;    int d=1;
    p=(TreeNode *)malloc(sizeof(TreeNode));
    p->entry=*pe;    p->left=NULL;    p->right=NULL;
    if (!pt->root)
        pt->root=p;
    else{
        curr=pt->root;
        while (curr) {
            prev=curr;
            if (LT (pe->key, curr->entry.key))
                curr=curr->left;
            else
                curr=curr->right;
            d++;
        }
        if (LT (pe->key, prev->entry.key))
            prev->left=p;
        else
            prev->right=p;
    }
    if (d>pt->depth)
        pt->depth=d;
    pt->size++;
}

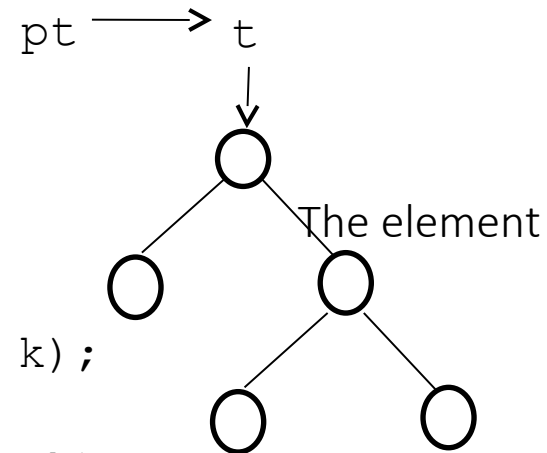
```



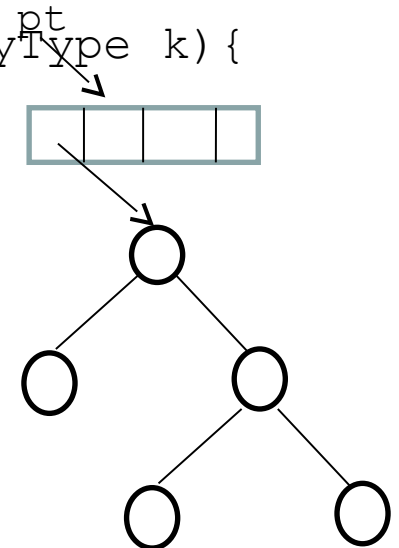
The red statements are the extra statments
when we write InsertTree

//Pre: Tree has been created and initialized (can be empty)

```
int FindItemTreeRec(Tree *pt, TreeEntry *pe, KeyType k){
    if (!*pt)
        return 0;
    if (EQ((*pt)->entry.key, k)){
        *pe=(*pt)->entry;
        return 1;
    }
    if (LT(k, (*pt)->entry.key))
        return FindItemTreeRec(&(*pt)->left, pe, k);
    else
        return FindItemTreeRec(&(*pt)->right, pe, k);
} //Very similar to Insert
```



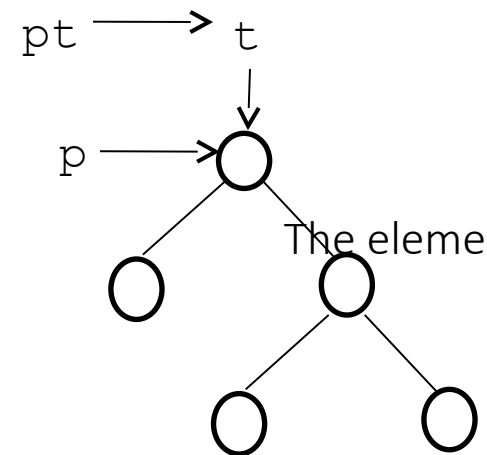
```
int FindItemTreeRec2(Tree2 *pt, TreeEntry *pe, KeyType k){
    return FindItemTreeRec(&pt->root, pe, k);
}
```



```

int FindItemTree(Tree *pt, TreeEntry *pe, KeyType k){
    int found=0;
    TreeNode *p=*pt;
    while(p && !(found=EQ(p->entry.key, k)))
        if (LT(k, p->entry.key))
            p=p->left;
        else
            p=p->right;
    if (found)
        *pe=p->entry;
    return found;
}

```



```

int FindItemTree2(Tree2 *pt, TreeEntry *pe, KeyType k){

    TreeNode *p=pt->root; //The only change
    ...
}

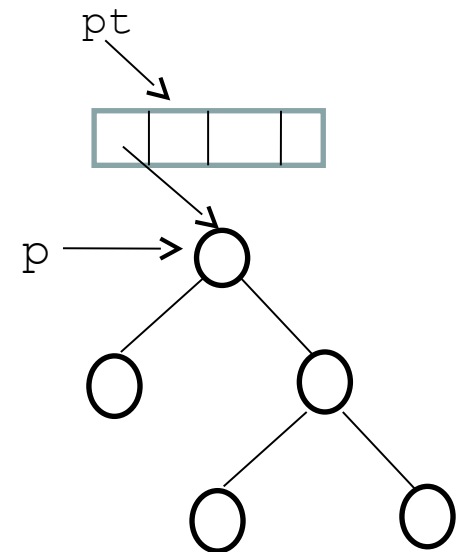
```

Note: The only precondition is that the tree has been created. If the tree is NULL the code still works; this is because “*Short circuit evaluation*”. However, It will cause runtime error (try it) if it is:

```

While (! (found=EQ (p->entry.key, k)) && p)

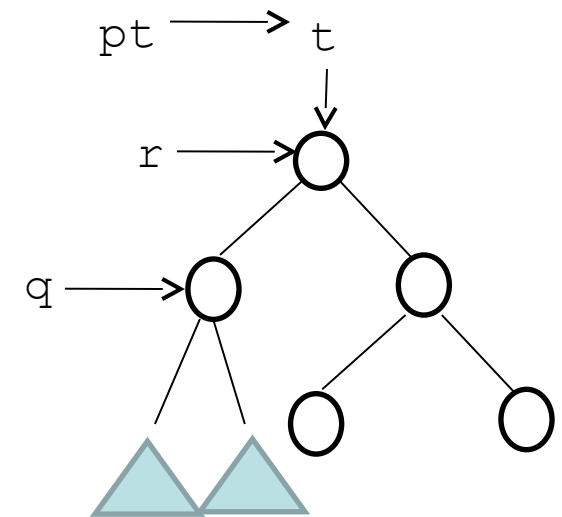
```



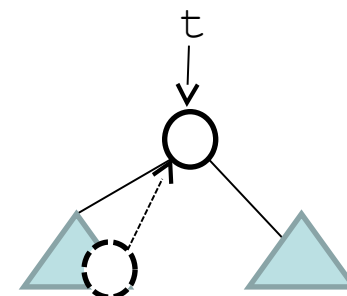
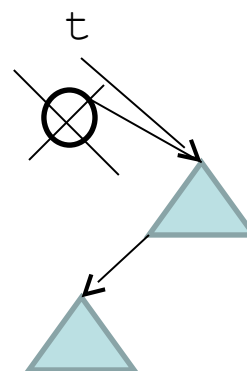
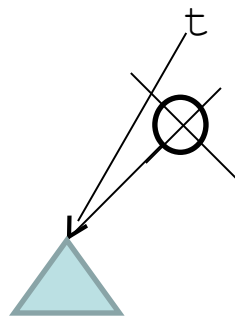
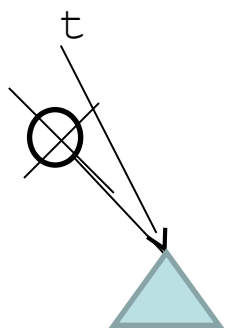
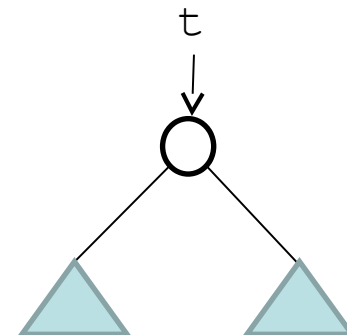
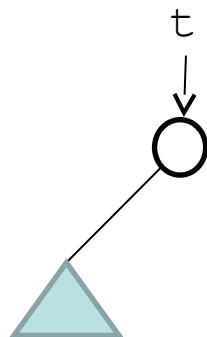
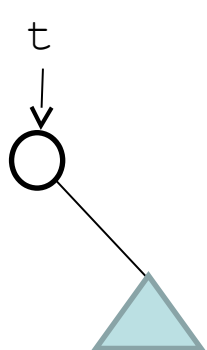
- To delete a node, first we need to find this particular node (search), then delete it.
- The algorithm is naturally iterative; we will no longer consider the recursive version.
- Then the problem is reduced to how to delete a particular node?

//Pre: Tree has been created and initialized (can be empty)

```
int DeleteItemTree(Tree *pt, TreeEntry *pe, KeyType k){
    int found=0;
    TreeNode *q=*pt, *r=NULL;
    while(q && !(found=EQ(k, q->entry.key))){
        r=q;
        if(LT(k, q->entry.key))
            q=q->left;
        else
            q=q->right;
    }
    if (found) {
        *pe=q->entry;
        if(!r) //Case of deleting the root
            DeleteNodeTree(pt);
        else if(LT(k, r->entry.key))
            DeleteNodeTree(&r->left);
        else
            DeleteNodeTree(&r->right);
    }
    return found;
}
```



To delete a node we have three cases; no problem in the first two of them:



Inefficient solution

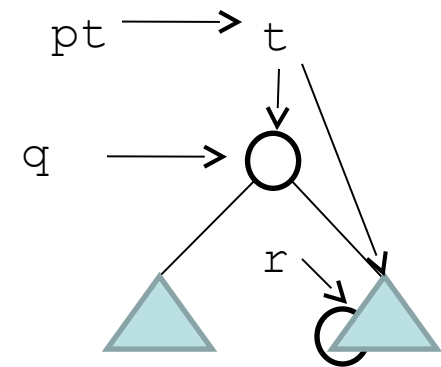
Efficient solution,

Very important: in `tree2`, updating `size` is trivial; i.e., `size--`. However, updating `depth` is tricky. We do not know whether deleting this node results in decreasing the depth or not. This may let the designer step back and decide not to include the field `depth` in `tree2`. This is a practical example of how the design of a data structure evolves (like any thing else).

```

void DeleteNodeTree(Tree *pt){//Very inefficient version
    TreeNode *q=*pt, *r;
    if(!(*pt)->left)//First case
        *pt=(*pt)->right;
    else if(!(*pt)->right)//Second case
        *pt=(*pt)->left;//Also, both account for a leaf node
    else{//third case
        *pt=(*pt)->right;
        for(r=q->right; r->left; r=r->left);
        r->left=q->left;
    }
    free(q);
}

```



In the efficient version, we change the body of the `else` statement; i.e., the third case.

```

void DeleteNodeTree(Tree *pt){
    TreeNode *q=*pt, *r;
    if(!(*pt)->left)//First case
        *pt=(*pt)->right;
    else if(!(*pt)->right)//Second case
        *pt=(*pt)->left;//Also, both account for a leaf node
    else{//third case
        q=(*pt)->left;
        if(!q->right){
            (*pt)->entry=q->entry;
            (*pt)->left=q->left;
        }
        else{
            do{
                r=q;
                q=q->right;
            }while(q->right);
            (*pt)->entry=q->entry;
            r->right=q->left;
        }
    }
    free(q);
}

```

